Lab 6 - Code and Branch Coverage

Atul Suresh | PES1UG24CS098 | Section B

Password Policy Checker - Scenario 39 | 03 November 2026

Objective: write separate Python test suites and achieve at least 90% statement coverage and 90% branch coverage of the supplied password policy module.

Test file	Tests passed	Statement coverage	Branch coverage
test_code_coverage.py	47 / 47	47 / 47 = **100%**	Not measured
test_branch_coverage.py	12 / 12	47 / 47 = **100%**	36 / 36 = **100%**

GitHub: Individual repository - Lab-6 folder | Assigned scenario

Separate commits: Code coverage history | Branch coverage history

1. Code coverage - 100%

```
Atul Suresh | PES1UG24CS098 | Lab 6 - Code Coverage
Assisted execution in Linux terminal | 03 November 2026

Python 3.12.14
+ python3 -m pytest test_code_coverage.py --cov=password_policy --cov-report=term-missing --cov-report=json:evidence/cod
e_coverage.json --cov-fail-under=90
============================================= test session starts ==============================================
platform linux -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: /workspace/scratch/3a64daa2bd86/se-labs/Lab-6
configfile: pytest.ini
plugins: cov-7.1.0
collected 47 items

test_code_coverage.py .............................................                                      [100%]

================================================ tests coverage ================================================
_____________________________________ coverage: platform linux, python 3.12.14-final-0 _____________________________________

Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
password_policy.py      47      0   100%
-------------------------------------------------
TOTAL                   47      0   100%
Coverage JSON written to file evidence/code_coverage.json
Required test coverage of 90% reached. Total coverage: 100.00%
============================================== 47 passed in 0.07s ==============================================
+ python3 show_coverage.py evidence/code_coverage.json
Line coverage:   47/47 (100.00%)
90% minimum: PASS
+ set +x
Execution complete. Press Enter to close.
```

The 47 tests check length boundaries (including 7/8 and 64/65), missing character categories, common-password matches, strength boundaries, fix messages and priority, Unicode/whitespace behavior, and independent calls. All 47 executable statements ran; none were missed.

Screenshots show actual assisted Linux terminal execution, using Python 3.12.14, pytest 9.2.0, pytest-cov 7.1.0 and coverage.py 7.17.0. Coverage is restricted to password_policy.py.

2. Branch coverage - 100%

A separate suite of 12 cases covers all 36 measured branch destinations.

```
Atul Suresh | PES1UG24CS098 | Lab 6 - Branch Coverage
Assisted execution in Linux terminal | 03 November 2026

Python 3.12.14
+ python3 -m pytest test_branch_coverage.py --cov=password_policy --cov-branch --cov-report=term-missing --cov-report=js
on:evidence/branch_coverage.json --cov-fail-under=90
============================================== test session starts ==============================================
platform linux -- Python 3.12.14, pytest-9.2.0, pluggy-1.7.0
rootdir: /workspace/scratch/3a64daa2bd86/se-labs/Lab-6
configfile: pytest.ini
plugins: cov-7.1.0
collected 12 items

test_branch_coverage.py ............                                                                    [100%]

================================================ tests coverage =================================================
_____________________________________ coverage: platform linux, python 3.12.14-final-0 _____________________________________

Name                Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
password_policy.py      47      0     36      0   100%
-----------------------------------------------------------------
TOTAL                   47      0     36      0   100%
Coverage JSON written to file evidence/branch_coverage.json
Required test coverage of 90% reached. Total coverage: 100.00%
============================================== 12 passed in 0.04s ===============================================
+ python3 show_coverage.py evidence/branch_coverage.json
Line coverage:   47/47 (100.00%)
Branch coverage: 36/36 (100.00%)
90% minimum: PASS
+ set +x
Execution complete. Press Enter to close.
```

Decision / path	Cases exercised
Length and common passwords	Too short, valid length, too long; common and non-common inputs.
Character classification and presence	Uppercase, lowercase, digit, special; each missing category; empty and nonempty loops.
Strength decisions	No issues: good / strong; one or two issues: weak; three or more: very weak.
Fix suggestions	No issue, too short, common, missing category, and too-long fallback.

Verification

Both suites pass: 47 code-coverage cases and 12 branch-coverage cases. Each required metric is checked separately against 90%. Full test files and reproduction commands follow.

The supplied password_policy.py is unchanged from upstream commit bbef87f. Raw test output and coverage data are in evidence/.

3. test_code_coverage.py - part 1

```
"""Boundary and branch tests for the supplied password policy."""

import pytest

from password_policy import check_password, strength, suggest_fix


def test_valid_password_has_no_issues():
    assert check_password("Abcdef1!") == []


def test_valid_password_is_good():
    assert strength("Abcdef1!") == "good"


@pytest.mark.parametrize(
    "password, expected",
    [
        ("", ["too short", "no uppercase", "no lowercase", "no digit", "no special"]),
        ("Aa1!abc", ["too short"]),
        ("Aa1!abcd", []),
        ("Aa1!abcde", []),
        ("Aa1!" + "a" * 59, []),
        ("Aa1!" + "a" * 60, []),
        ("Aa1!" + "a" * 61, ["too long"]),
    ],
    ids=["empty", "length-7", "length-8", "length-9", "length-63", "length-64", "length-65"],
)
def test_password_length_boundaries(password, expected):
    assert check_password(password) == expected


@pytest.mark.parametrize(
    "password, expected",
    [
        ("abcdef1!", ["no uppercase"]),
        ("ABCDEF1!", ["no lowercase"]),
        ("Abcdefg!", ["no digit"]),
        ("Abcdef12", ["no special"]),
    ],
    ids=["uppercase", "lowercase", "digit", "special"],
)
def test_each_missing_character_category(password, expected):
    assert check_password(password) == expected


@pytest.mark.parametrize(
    "password, expected",
    [
        ("password", ["common", "no uppercase", "no digit", "no special"]),
        ("PASSWORD", ["common", "no lowercase", "no digit", "no special"]),
        ("PaSsWoRd", ["common", "no digit", "no special"]),
        ("123456", ["too short", "common", "no uppercase", "no lowercase", "no special"]),
        ("qwerty", ["too short", "common", "no uppercase", "no digit", "no special"]),
        ("letmein", ["too short", "common", "no uppercase", "no digit", "no special"]),
    ],
    ids=["password", "uppercase-common", "mixed-case-common", "digits-common", "qwerty", "letmein"],
)
def test_common_passwords_and_issue_order(password, expected):
    assert check_password(password) == expected


def test_common_word_with_extra_characters_is_not_an_exact_common_match():
    assert check_password("Password1!") == []
```

3. test_code_coverage.py - part 2

```
@pytest.mark.parametrize(
    "password, expected",
    [
        ("Aa1!abc", "weak"),
        ("Aa1!abcd", "good"),
        ("Aa1!" + "a" * 9, "good"),
        ("Aa1!" + "a" * 10, "strong"),
        ("Aa1!" + "a" * 60, "strong"),
        ("Aa1!" + "a" * 61, "weak"),
        ("abcdef1!", "weak"),
        ("abcdef12", "weak"),
        ("abcdefgh", "very weak"),
        ("", "very weak"),
    ],
    ids=["length-7", "length-8", "length-13", "length-14", "length-64", "length-65", "one-issue", "two-issues",
        "three-issues", "empty"],
)
def test_strength_boundaries_and_issue_counts(password, expected):
    assert strength(password) == expected


@pytest.mark.parametrize(
    "issues, expected",
    [
        ([], "none needed"),
        (["too short"], "add more characters"),
        (["common"], "avoid common passwords"),
        (["no uppercase"], "add uppercase"),
        (["no lowercase"], "add lowercase"),
        (["no digit"], "add digit"),
        (["no special"], "add special"),
        (["too long"], "shorten it"),
        (["unrecognized issue"], "shorten it"),
    ],
    ids=["none", "too-short", "common", "uppercase", "lowercase", "digit", "special", "too-long", "fallback"],
)
def test_fix_for_each_issue(issues, expected):
    assert suggest_fix(issues) == expected


@pytest.mark.parametrize(
    "password, expected",
    [
        ("123456", "add more characters"),
        ("password", "avoid common passwords"),
        ("abcdefgh", "add uppercase"),
    ],
    ids=["length-before-common", "common-before-category", "first-missing-category"],
)
def test_fix_uses_first_issue_from_checker_without_changing_the_list(password, expected):
    issues = check_password(password)
    original = issues.copy()
    assert suggest_fix(issues) == expected
    assert issues == original


@pytest.mark.parametrize(
    "password",
    ["Abcdef1 ", "Abcdef1\n", "\u00c9bcdef1!", "Abcdef\u0661!"],
    ids=["space-is-special", "newline-is-special", "unicode-uppercase", "unicode-digit"],
)
def test_existing_whitespace_and_unicode_classification(password):
    # These document the supplied Python character-classification behavior.
    assert check_password(password) == []


def test_results_are_independent_between_calls():
    first = check_password("Abcdef1!")
    first.append("caller change")
    assert check_password("Abcdef1!") == []
```

4. test_branch_coverage.py

```
"""Exercise both outcomes of the policy's decisions independently."""

import pytest

from password_policy import check_password, strength, suggest_fix


@pytest.mark.parametrize(
    "password, issues, rating, fix",
    [
        ("Abcdef1!", [], "good", "none needed"),
        ("Aa1!" + "a" * 10, [], "strong", "none needed"),
        ("Ab1!", ["too short"], "weak", "add more characters"),
        ("Aa1!" + "a" * 61, ["too long"], "weak", "shorten it"),
        ("password", ["common", "no uppercase", "no digit", "no special"], "very weak", "avoid common passwords"),
        ("abcdef1!", ["no uppercase"], "weak", "add uppercase"),
        ("ABCDEF1!", ["no lowercase"], "weak", "add lowercase"),
        ("Abcdefg!", ["no digit"], "weak", "add digit"),
        ("Abcdef12", ["no special"], "weak", "add special"),
        ("abcdef12", ["no uppercase", "no special"], "weak", "add uppercase"),
        ("abcdefgh", ["no uppercase", "no digit", "no special"], "very weak", "add uppercase"),
        ("", ["too short", "no uppercase", "no lowercase", "no digit", "no special"], "very weak", "add more characters"),
    ],
    ids=[
        "valid-good", "valid-strong", "too-short", "too-long", "common",
        "missing-uppercase", "missing-lowercase", "missing-digit",
        "missing-special", "two-issues", "three-issues", "empty-loop",
    ],
)
def test_policy_decision_paths(password, issues, rating, fix):
    assert check_password(password) == issues
    assert strength(password) == rating
    assert suggest_fix(issues) == fix
```

Separate coverage commands

Run these from the Lab-6 folder after installing requirements.txt.

```
python3 -m pip install -r requirements.txt

python3 -m pytest test_code_coverage.py \
  --cov=password_policy --cov-report=term-missing \
  --cov-report=json:evidence/code_coverage.json --cov-fail-under=90
python3 show_coverage.py evidence/code_coverage.json

python3 -m pytest test_branch_coverage.py \
  --cov=password_policy --cov-branch --cov-report=term-missing \
  --cov-report=json:evidence/branch_coverage.json --cov-fail-under=90
python3 show_coverage.py evidence/branch_coverage.json
```